

Assignment-1

1. What is Full Stack Development?

Answer:

Full Stack Development is the process of developing both the **Frontend** and **Backend** parts of a web application. A Full Stack Developer can work on the complete application, including user interface, server-side logic, APIs, and databases.

Example: Amazon, Facebook, and Instagram are full-stack applications because they have a frontend, backend, and database.

2. What is Frontend?

Answer:

Frontend is the part of a website or application that users can see and interact with directly. It is responsible for the appearance and user experience of a website.

Technologies Used:

- HTML
- CSS
- JavaScript
- React.js
- Angular
- Vue.js

Example: The Amazon homepage, search bar, product cards, and buttons are all part of the frontend.

3. What is Backend?

Answer:

Backend is the server-side part of a web application. It handles business logic, user authentication, data processing, and communication with the database.

Functions of Backend:

- Processing user requests
- Managing authentication and authorization
- Handling APIs
- Connecting with databases

Technologies Used:

- Java (Spring Boot)
- Node.js
- Python (Django/Flask)
- PHP

Example: When a user logs into Amazon, the backend verifies the username and password.

4. What is a Database?**Answer:**

A Database is an organized collection of data that can be stored, managed, and retrieved efficiently. Databases help applications store information permanently.

Uses of Database:

- Store user information
- Store product details
- Store order history
- Store payment records

Examples:

- MySQL
- PostgreSQL
- MongoDB
- Oracle Database

Example: Amazon stores product information, customer details, and order history in databases.

5. What is DNS?

Answer:

DNS (Domain Name System) is a system that converts human-readable domain names into IP addresses. It acts like the phonebook of the internet.

How DNS Works:

1. User enters a website name (e.g., google.com).
2. DNS finds the corresponding IP address.
3. The browser connects to the correct server.
4. The website is displayed to the user.

Example:

- Domain Name: google.com
- IP Address: 142.250.xxx.xxx

Without DNS, users would need to remember numerical IP addresses instead of website names.

6. What is an IP Address?

Answer:

An IP (Internet Protocol) Address is a unique numerical identifier assigned to every device connected to a network. It helps devices communicate with each other over the internet.

Types of IP Addresses:

1. IPv4 (Example: 192.168.1.1)
2. IPv6 (Example: 2001:0db8:85a3::8a2e:0370:7334)

Example:

When you visit a website, your browser connects to the server using its IP address

7. What is Client-Server Architecture?

Answer:

Client-Server Architecture is a model in which a client sends requests and a server processes those requests and returns responses.

Components:

- Client: Browser or mobile app
- Server: Processes requests
- Database: Stores information

Example:

When you search for a product on Amazon:

1. Browser sends a request.
2. Server processes the request.
3. Database retrieves product information.
4. Results are displayed on the browser.

Diagram:

Client → Server → Database
← Response ←

8. What is an API?

Answer:

API (Application Programming Interface) is a set of rules that allows different software applications to communicate with each other.

Uses of APIs:

- Data sharing
- Payment gateways
- Social media login
- Weather information

Example:

When a weather app displays temperature data, it gets the information through a Weather API.

Advantages:

- Faster development
- Easy integration
- Better scalability

9. What are HTTP Methods?**Answer:**

HTTP Methods define the type of action that should be performed on a resource.

Method Purpose

GET Retrieve data

POST Create new data

PUT Update existing data

DELETE Remove data

Examples:**GET**

Get all products from database

POST

Add a new product

PUT

Update product details

DELETE

Delete a product

These methods are commonly used in REST APIs.

10. What are Developer Tools?

Answer:

Developer Tools are built-in browser tools that help developers inspect, debug, and analyze websites.

Three Important Tabs:

1. Elements Tab

- Used to inspect HTML structure and CSS styles.
- Helps developers understand how a webpage is designed.

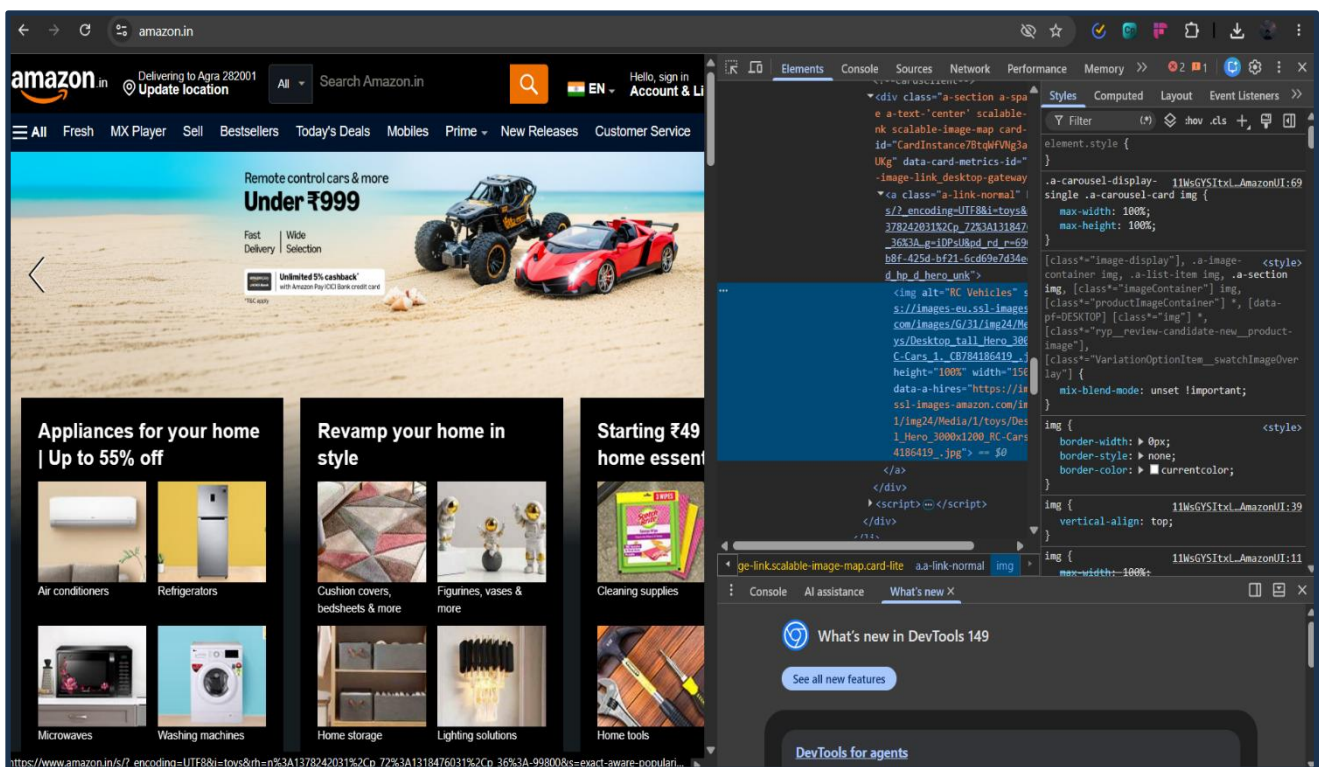
2. Console Tab

- Used to view JavaScript errors and messages.
- Helps in debugging code.

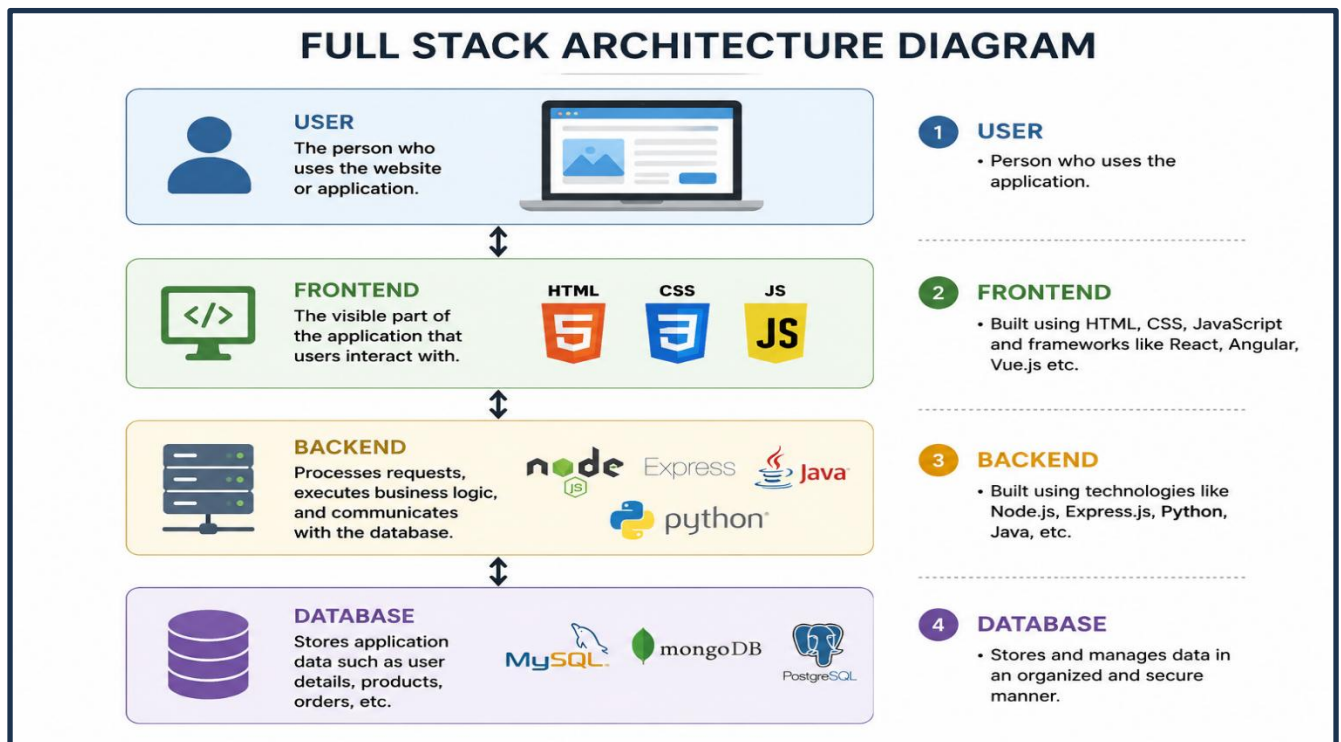
3. Network Tab

- Used to monitor requests and responses between browser and server.
- Helps analyze website performance.

Example Screenshot



11. Draw the Full Stack Architecture Diagram Short Description of Each Layer



1. User

- The user is the person who interacts with the website or application.
- The user sends requests and receives information through the frontend.

2. Frontend

- Frontend is the visible part of the application.
- It is developed using HTML, CSS, JavaScript, React, Angular, etc.
- It collects user input and displays data.

3. Backend

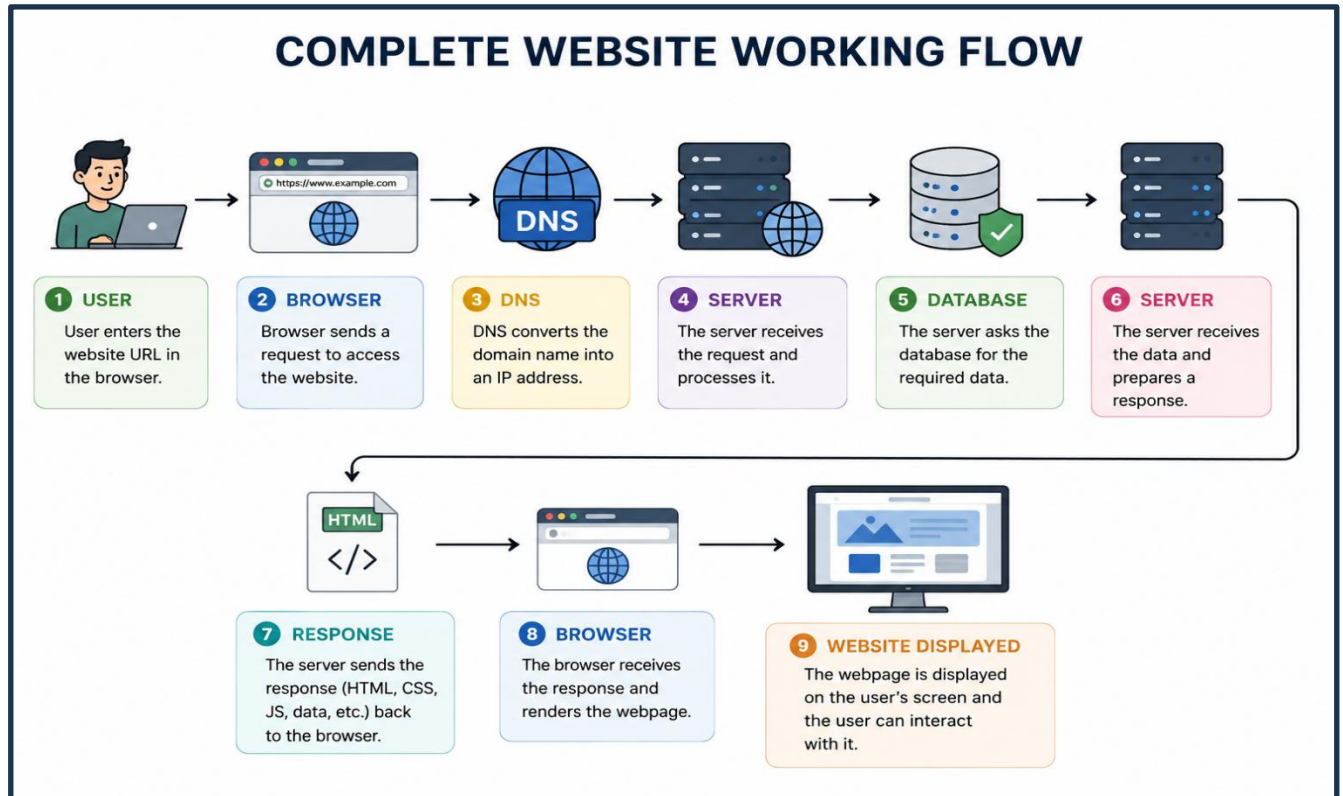
- Backend processes user requests and executes business logic.
- It handles authentication, APIs, and communication with the database.
- Common technologies include Node.js, Java, Python, and Express.js.

4. Database

- Database stores and manages application data.
- It contains information such as user accounts, products, orders, and transactions.

- Examples include MySQL, MongoDB, and PostgreSQL.

12. Draw the Complete Website Working Flow



1. User

The user enters a website URL (for example, www.amazon.in) in the browser.

2. Browser

The browser sends a request to access the website.

3. DNS (Domain Name System)

DNS converts the website name into an IP address so the browser can find the correct server.

4. Server

The server receives the request and processes it.

5. Database

The server requests the required information from the database.

6. Server

The database sends the data back to the server, and the server prepares a response.

7. Response

The server sends the response (HTML, CSS, JavaScript, images, etc.) to the browser.

8. Browser

The browser receives the response and renders the webpage.

9. Website Displayed

The webpage appears on the user's screen, and the user can interact with it.

13. Student Registration System

A student fills a registration form on a website.

What happens in the Frontend?

- The student opens the registration page in the browser.
- The student enters details such as Name, Email, Phone Number, and Course.
- The frontend validates the input fields and checks for missing or incorrect information.
- After successful validation, the form data is sent to the backend server.

What happens in the Backend?

- The backend receives the registration data from the frontend.
- It verifies and validates the submitted information.
- It checks whether the student is already registered.
- If the data is valid, the backend processes the request and sends the data to the database.
- A success message is generated and returned to the frontend.

What happens in the Database?

- The database stores the student's information permanently.
- A new student record is created.
- The data can be retrieved later for academic and administrative purposes.
- The database ensures secure and organized storage of information.

14. Amazon Search Scenario

A user searches for "Laptop" on Amazon.

Complete Flow

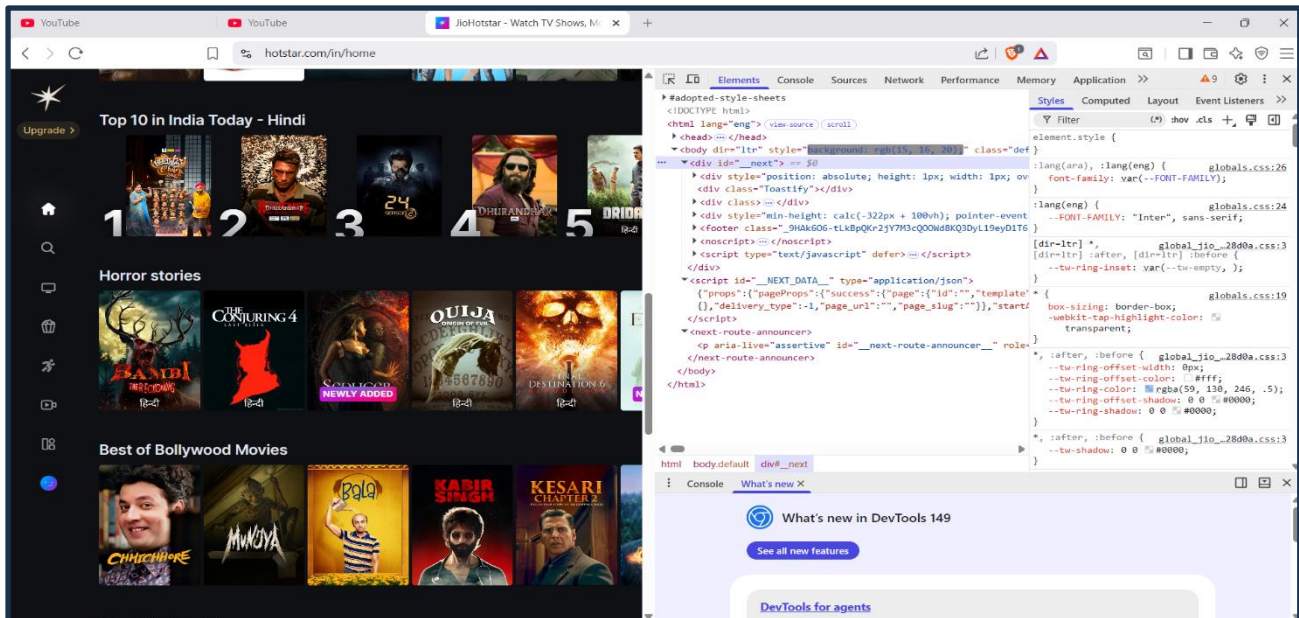
1. The user opens Amazon in a **Browser**.
2. The user enters the keyword "**Laptop**" in the search bar.
3. The **Browser** sends the search request to the **Server**.
4. The **Server** receives and processes the request.
5. The **Server** searches the **Database** for products related to laptops.
6. The **Database** returns matching product information to the server.
7. The **Server** prepares a **Response** containing the search results.
8. The **Response** is sent back to the **Browser**.
9. The **Browser** displays the list of laptops to the user.



15. Website Observation Activity

Website Name: JioHotstar.com

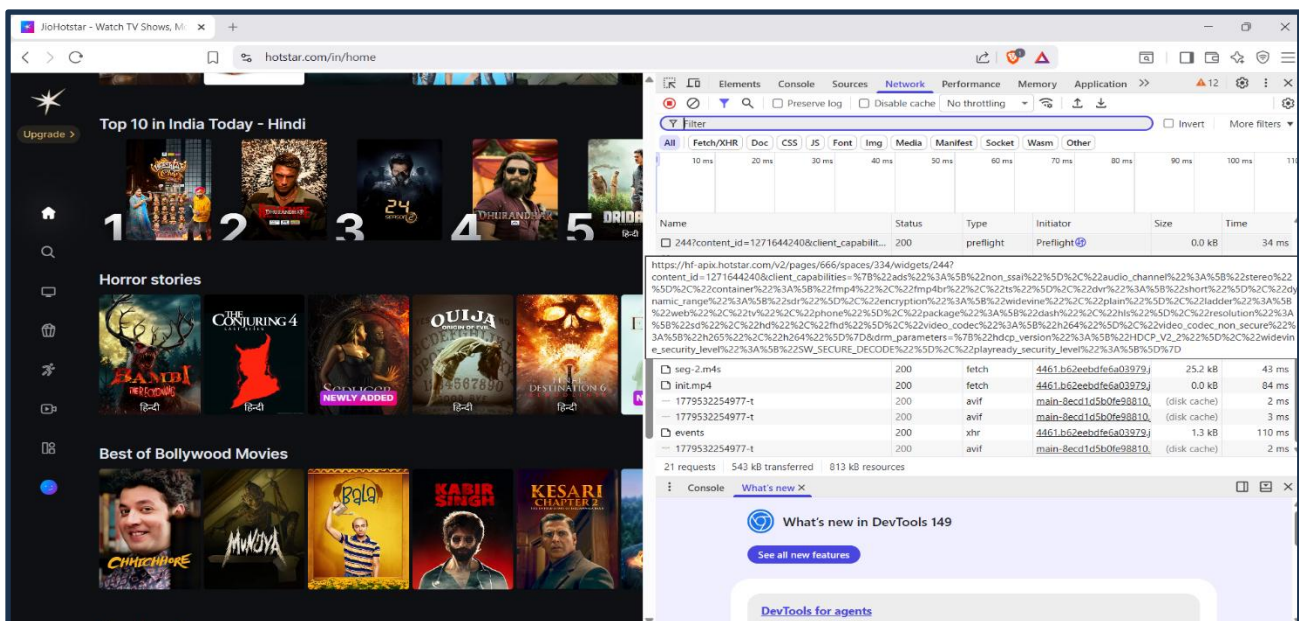
Figure 1: Elements Tab



Observation from Console Tab:

- The Console tab displays JavaScript messages, warnings, and errors.
- It helps developers debug and test web applications.
- In the screenshot, player-related logs and warning messages are visible.
- Developers use this tab to identify and fix issues in the website.

Figure 3: Network Tab



Observation from Network Tab:

- The Network tab shows all requests and responses exchanged between the browser and the server.
- It displays files such as images, videos, scripts, and API requests.
- The tab also provides information about request status, size, and loading time.
- It is useful for analyzing website performance and troubleshooting loading issues.

16. In your own words, explain how a website works from the moment a user enters a URL until the webpage appears on the screen.

Answer:

When a user enters a website address (URL) in the browser, the browser sends a request to find the website. The request goes to the server where the website is stored. The server collects the required information and sends it back to the browser. The browser then shows the webpage on the screen. This process happens within a few seconds.

Example:

When you type www.amazon.in in your browser, Amazon's server sends the webpage data, and the browser displays the Amazon website.

17. Explain the difference between Frontend, Backend, and Database using a real-world example.

Example: Amazon

Frontend

Frontend is the part of the website that users can see and use.

Examples:

- Search bar
- Product images
- Add to Cart button
- Product details

The frontend helps users interact with the website.

Backend

Backend works behind the scenes.

Examples:

- Checking user login details
- Processing orders
- Managing payments
- Handling search requests

The backend makes sure everything works correctly.

Database

The database stores information.

Examples:

- User accounts
- Product information
- Order history
- Customer details

The database keeps all the data safe and organized.